

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 01 -

FUNDAMENTOS DE CONTAINERS E VIRTUALIZAÇÃO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	15
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17

EMANIP

O QUE VEM POR AÍ?

Ao longo dos últimos anos, Machine Learning deixou de ser uma atividade restrita à experimentação local e passou a ocupar um papel central em produtos, serviços e decisões de negócio. Modelos que antes rodavam apenas em notebooks de pesquisadores hoje precisam ser treinados em ambientes distribuídos, implantados em produção, escalados conforme a demanda e monitorados continuamente.

Nesse contexto, surge um desafio recorrente: como garantir que um modelo funcione da mesma forma em diferentes ambientes? É comum que um experimento apresente ótimos resultados no computador do(a) cientista de dados, mas falhe ao ser executado em outro servidor, em um cluster ou em produção. Diferenças de sistema operacional, versões de bibliotecas, drivers ou configurações de hardware são causas frequentes desse problema. É exatamente nesse ponto que containers se tornam fundamentais.

Nesta aula, você começará a construir a base conceitual que sustenta todo o restante do curso. Vamos entender por que a virtualização surgiu, como containers se diferenciam de máquinas virtuais, quais mecanismos do sistema operacional tornam containers possíveis e por que essas decisões técnicas impactam diretamente o ciclo de vida de modelos de Machine Learning.

Ao longo da aula, você será convidado(a) a mudar a forma como enxerga o ambiente de execução de um modelo. Em vez de tratá-lo como um detalhe operacional, passaremos a vê-lo como parte integrante do experimento e do produto de ML.

Você também começará a perceber que containers não são apenas uma ferramenta técnica, mas um elemento de integração entre pesquisa, engenharia e produção. Eles criam um contrato claro sobre como o modelo deve ser executado, reduzindo ambiguidades, retrabalho e falhas no caminho até a produção.

Essa base conceitual será essencial para compreender, nas próximas aulas, como Docker empacota aplicações, como Kubernetes orquestra containers em escala e como tudo isso se conecta a práticas modernas de MLOps.

HANDS ON

Nesta aula prática, o foco não está em memorizar comandos ou dominar ferramentas imediatamente, mas em compreender como o ambiente de execução influencia aplicações de Machine Learning. Antes mesmo de estudarmos Docker em profundidade, você irá observar, um problema clássico enfrentado por times de dados e engenharia: a inconsistência entre ambientes.

É muito comum que um experimento funcione perfeitamente no computador de quem o desenvolveu, mas apresente falhas ao ser executado em outro servidor, em um cluster ou em produção. Diferenças de sistema operacional, versões de bibliotecas, arquitetura de hardware ou configurações locais são suficientes para gerar erros difíceis de diagnosticar.

Comece observando o ambiente local em que você está executando os comandos. Ao rodar `python --version`, você identifica qual versão do Python está disponível no seu sistema. Em seguida, ao executar `python -c "import platform; print(platform.platform())"`, você obtém informações sobre o sistema operacional. Esses dois comandos simples já revelam algo importante: o ambiente não é neutro. Ele carrega decisões técnicas que impactam diretamente qualquer código de Machine Learning.

Agora, execute comandos semelhantes, mas dentro de um container. Ao rodar `docker run --rm python:3.11-slim python --version`, você perceberá que a versão do Python exibida é sempre a mesma, independentemente da versão instalada na sua máquina. O mesmo ocorre ao executar `docker run --rm python:3.11-slim python -c "import platform; print(platform.platform())"`. O sistema operacional reportado é consistente, pois o código está sendo executado em um ambiente padronizado definido pela imagem do container.

Neste ponto, vale fazer uma pausa para refletir. Você não instalou manualmente o Python, não configurou bibliotecas e não alterou o seu sistema operacional. Ainda assim, executou um ambiente previsível e controlado. Isso acontece porque o container encapsula o runtime, as dependências e parte do contexto de execução, isolando o processo do restante do sistema. Esse é o primeiro

contato prático com o conceito de reprodutibilidade, que será central ao longo de todo o curso.

Para reforçar esse entendimento, execute o comando `docker images` e observe a lista de imagens disponíveis localmente. Em seguida, utilize `docker ps -a` para visualizar containers que já foram executados.

EMANIP

SAIBA MAIS

É comum apresentar containers como uma “forma moderna de empacotar aplicações”. Embora essa definição não esteja errada, ela é incompleta. Containers são, na prática, uma abstração de sistemas operacionais aplicada ao nível da aplicação. Eles deslocam parte da responsabilidade tradicional da infraestrutura para o próprio artefato entregue, redefinindo a fronteira entre software e ambiente de execução.

Ao encapsular runtime, bibliotecas e configurações, o container cria um ambiente determinístico, reduzindo o acoplamento entre aplicação e infraestrutura. Em Machine Learning, isso é especialmente crítico, pois o comportamento de modelos depende fortemente de detalhes do ambiente — desde versões de bibliotecas matemáticas até drivers de hardware e características do sistema operacional.

Sob essa perspectiva, containers não devem ser vistos apenas como um mecanismo de deploy, mas como um elemento estrutural do design de sistemas de ML, influenciando arquitetura, governança e operação ao longo de todo o ciclo de vida do modelo.

O problema de base: por que "funciona na minha máquina" ainda custa caro em ML

Projetos de Machine Learning combinam código de negócio, bibliotecas numéricas, frameworks de treino e runtime de inferência. Cada um desses componentes tem versão, dependência transiente e comportamento específico de sistema operacional. Quando esse conjunto não é controlado de forma rigorosa, o time paga com retrabalho: pipelines quebram, previsões divergem entre ambientes e deploys se tornam lentos por medo de regressão.

Em software tradicional, essa dor já era relevante. Em ML, ela aumenta porque, além do código, existe artefato de modelo, dependência de hardware (CPU/GPU), bibliotecas nativas e requisitos de desempenho. Por isso, falar de containers não é apenas falar de empacotamento. É falar de confiabilidade do processo de entrega de modelos e da capacidade de escalar inferência com consistência.

Virtualização tradicional: o papel do hypervisor

A virtualização baseada em hypervisor foi um marco histórico. A proposta é executar várias máquinas virtuais (VMs) no mesmo hardware físico, cada uma com seu próprio sistema operacional convidado. Na prática, uma VM se comporta como um computador completo: kernel próprio, espaço de usuário próprio e conjunto de drivers correspondente.

Existem duas categorias clássicas. O hypervisor tipo 1 (bare metal) executa diretamente sobre o hardware e, normalmente, entrega melhor isolamento e controle para data centers. O hypervisor tipo 2 executa sobre um sistema operacional host e é muito comum em ambientes de desenvolvimento. Em ambos os casos, o ganho principal é consolidação de infraestrutura e isolamento forte entre cargas.

Mas VMs possuem custo operacional. Cada instância carrega um sistema operacional completo, consumindo mais memória e armazenamento. O tempo de boot tende a ser maior e a densidade por host tende a ser menor quando comparada à de containers. Em ML, esse overhead pode dificultar ciclos rápidos de experimento e release, principalmente quando o time precisa testar múltiplas variações de serviços de inferência por dia.

Containers: virtualização em nível de sistema operacional

Containers seguem outra estratégia: em vez de virtualizar hardware, virtualizam o espaço de execução no nível do sistema operacional. Isso significa que os processos no container compartilham o kernel do host, mas são isolados em relação a processos, rede, recursos e sistema de arquivos por mecanismos do próprio kernel Linux.

Essa mudança de modelo explica por que containers iniciam mais rápido e usam menos recursos. Não existe boot de um sistema operacional convidado completo para cada instância. O que existe é um processo isolado, com seu próprio contexto de runtime. Para times de ML, isso acelera muito tarefas como testes de serviço de inferência, reprodução de bugs e promoção entre ambientes (dev, homologação e produção).

A comparação correta não é "containers são melhores que VMs em tudo". A pergunta certa é: qual nível de isolamento, desempenho e flexibilidade eu preciso para este caso? Em workloads regulados ou multi-tenant extremos, VMs podem ser preferíveis em partes da arquitetura. Em pipelines e serviços de ML com alta iteração, containers geralmente oferecem melhor equilíbrio entre segurança, velocidade e portabilidade.

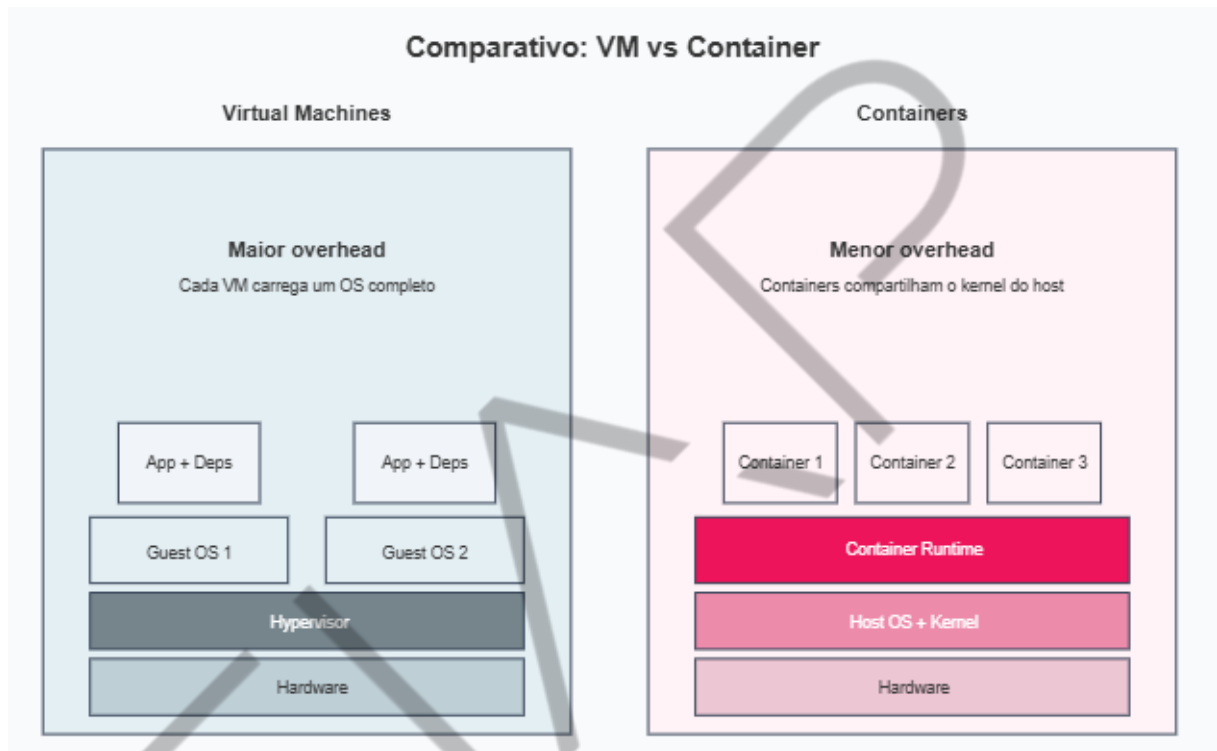


Figura 1 - Comparação visual entre pilha de virtualização com VMs e com containers
Fonte: Google Imagens (2026)

Essa diferença tem implicações diretas de segurança e governança. Em containers tradicionais, uma vulnerabilidade no kernel pode potencialmente afetar todos os workloads em execução no host. Por esse motivo, ambientes corporativos e multi-tenant exigem uma postura de segurança mais rigorosa, combinando containers com políticas, hardening e monitoramento contínuo.

Em Machine Learning, essa sensibilidade aumenta quando:

- **Há execução de código de múltiplos times:** pipelines e experimentos diferentes compartilham a mesma infraestrutura, ampliando o impacto de falhas de isolamento.

- **Modelos são treinados ou executados com dados sensíveis:** dados pessoais, financeiros ou regulados exigem controles mais rígidos de acesso e segregação.
- **Plataformas oferecem ML como serviço interno ou externo:** ambientes compartilhados, como notebooks ou APIs de inferência, aumentam a superfície de ataque e exigem isolamento reforçado.

A segurança em containers não é obtida por um único mecanismo, mas por um conjunto de camadas de defesa que, juntas, reduzem riscos e aumentam previsibilidade. Em ambientes de ML, em que pipelines automatizados executam código com alta frequência, negligenciar essas camadas pode transformar containers em um vetor de risco significativo.

Algumas práticas fundamentais incluem:

- **Princípio do menor privilégio:** containers devem executar com usuários não-root sempre que possível. Executar processos privilegiados aumenta o risco de escalonamento de permissões e exploração de vulnerabilidades no host.
- **Redução de capacidades do kernel:** o kernel Linux oferece diversas capacidades sensíveis. Limitar syscalls e permissões disponíveis ao container reduz drasticamente o impacto de um eventual comprometimento.
- **Namespaces de usuário:** o uso de namespaces de usuário permite mapear usuários do container para IDs não privilegiados no host, adicionando uma camada extra de proteção mesmo quando processos possuem privilégios internos.
- **Imutabilidade de imagens:** containers devem ser tratados como artefatos imutáveis. Qualquer alteração no ambiente deve gerar uma nova imagem versionada, facilitando auditoria, rollback e rastreabilidade — aspectos essenciais em ambientes de ML regulados.

Namespaces são o mecanismo que entregam a "sensação" de ambiente próprio para cada container. Eles limitam o que um processo enxerga do sistema. Um processo no container pode acreditar que é o PID 1 daquele universo isolado, sem enxergar a árvore completa de processos do host.

Os tipos mais usados são:

- ``pid``: isola árvore de processos.
- ``net``: isola interfaces e pilha de rede.
- ``mnt``: isola pontos de montagem de filesystem.
- ``uts``: isola hostname e domínio.
- ``ipc``: isola mecanismos de comunicação entre processos.
- ``user``: mapeia IDs de usuário e grupo.

Para ML em produção, ``net`` e ``mnt`` são especialmente sensíveis. Um serviço de inferência pode depender de rotas específicas, portas, volumes e caminhos de artefatos. Sem isolamento bem definido, conflitos entre workloads surgem rapidamente.

cgroups: controle e contabilização de recursos

Se namespaces dizem "o que um processo enxerga", cgroups dizem "quanto ele pode consumir". Eles permitem limitar, priorizar e medir uso de CPU, memória, I/O e outros recursos por grupo de processos. Em ambientes de inferência, isso é fundamental para evitar que uma carga degrade o host inteiro.

Dois exemplos práticos:

1. Limitar memória de um serviço de inferência para evitar que picos de requisição matem processos vizinhos.
2. Definir cotas de CPU para manter previsibilidade de latência quando há vários modelos compartilhando o mesmo node.

Sem cgroups, a discussão de multitenancy e qualidade de serviço fica fraca. Com cgroups, fica possível impor contratos operacionais. Em termos de negócio, isso se traduz em menos indisponibilidade e melhor uso de infraestrutura.

Sistema de arquivos em camadas: por que imagens são rápidas e portáteis

Imagens de container são compostas por camadas imutáveis empilhadas. Cada instrução de build (por exemplo, instalação de dependência) gera uma nova camada. Isso habilita cache eficiente: se uma camada não mudou, ela pode ser reaproveitada em novos builds.

Para ML, esse modelo é valioso porque bibliotecas pesadas (como frameworks numéricos) podem ficar em camadas estáveis, enquanto o código da aplicação muda com mais frequência em camadas superiores. O resultado é build mais rápido, transferência menor entre ambientes e rollback mais previsível.

Também existe uma camada de escrita no runtime do container, geralmente efêmera. Ou seja: gravar arquivos temporários dentro do container não significa persistir dados para sempre. Esse detalhe é crítico para não perder artefatos, logs e resultados de inferência se a instância for recriada.

Do ponto de vista de performance, containers apresentam comportamentos distintos dependendo do tipo de workload.

- **Workloads CPU-bound:** em aplicações predominantemente baseadas em CPU, a sobrecarga introduzida por containers é mínima. Na maioria dos casos, o desempenho é praticamente equivalente ao da execução nativa.
- **Workloads GPU-bound:** em workloads que utilizam GPU, o cenário é mais complexo. O desempenho depende fortemente de:
 - Drivers corretos no host.
 - Compatibilidade entre runtime do container e hardware.
 - Gerenciamento adequado da memória da GPU.

Containers não virtualizam GPUs da mesma forma que CPUs. Eles expõem diretamente o dispositivo ao processo, o que torna o isolamento de GPU mais frágil e exige ferramentas adicionais. Em ML Engineering, isso afeta diretamente:

- Treinamento distribuído.
- Inferência em larga escala.
- Compartilhamento eficiente de GPUs.

Embora containers melhorem significativamente a reprodutibilidade, eles não eliminam todos os fatores de variação. Diferenças de hardware, paralelismo, bibliotecas matemáticas e ordem de execução podem gerar resultados ligeiramente distintos, especialmente em modelos complexos.

Reprodutibilidade é a capacidade de obter o mesmo comportamento dado o mesmo código, dependências, configurações e dados de entrada. Portabilidade é a capacidade de mover esse pacote entre ambientes mantendo previsibilidade. Os containers ajudam nas duas dimensões, mas não resolvem tudo sozinhos.

Para garantir reprodução real em ML, você precisa combinar:

- Imagem versionada.
- Dependências travadas.
- Configurações externas controladas (variáveis e secrets).
- Versão do modelo e metadados de treino.
- Observabilidade para detectar drift de comportamento.

No fluxo de MLOps, a containerização aparece como ponte entre ciência de dados e operação. O(a) cientista consegue empacotar um runtime consistente; o(a) engenheiro(a) de plataforma consegue implantar com padrão, políticas e monitoramento. Essa abordagem é essencial para auditoria, validação científica e governança de modelos.

Ao longo do ciclo de vida de ML, containers aparecem de forma recorrente:

- **No desenvolvimento**, padronizam ambientes e reduzem problemas de onboarding.
- **No treinamento**, garantem que jobs sejam executados de forma consistente em diferentes executores.
- **Na inferência**, servem como unidade básica de deploy e escala.
- **Na operação**, facilitam rollback, observabilidade e automação.

Essa presença contínua reforça a ideia de que containers são o artefato central do ML Engineering moderno.



Figura 2 – Containers como contrato técnico e organizacional
Fonte: Google Imagens (2026)

Além do aspecto técnico, containers desempenham um papel organizacional crucial. Eles formalizam um contrato claro entre pesquisa, engenharia e operações, reduzindo ambiguidades e conflitos.

Quando um modelo é entregue como container, não há espaço para interpretações sobre dependências, versões ou configuração. Isso acelera o caminho da pesquisa à produção e estabelece uma base sólida para práticas de MLOps.

Apesar de suas vantagens, containers não são uma solução universal. Overcommit de recursos, imagens mal construídas, ausência de políticas de segurança e falta de observabilidade podem criar novos pontos de falha.

Um indivíduo engenheiro de ML maduro não pergunta apenas “como usar containers?”, mas “quando, por que e com quais trade-offs?”. Essa capacidade de avaliação crítica diferencia o uso tático do uso arquitetural da tecnologia.

Erros comuns e como evitar

Erro: acreditar que container garante reprodução total sem controle de versão de dados e modelo.

Correção: versionar também artefatos, schema de entrada e configurações.

Erro: confundir isolamento de processo com segurança completa.

Correção: aplicar princípio de menor privilégio, imagens mínimas e varredura de vulnerabilidades.

Erro: gravar estado crítico na camada efêmera do container.

Correção: usar volumes e armazenamento persistente para dados que não podem ser perdidos.

Erro: ignorar observabilidade no início.

Correção: definir métricas de serviço (latência, erro, throughput) e métricas de modelo (acurácia operacional, drift) desde o primeiro deploy.

Tudo o que será estudado nas próximas aulas — Dockerfiles, Kubernetes, escalabilidade, CI/CD, MLOps e observabilidade — parte do pressuposto de que containers são compreendidos como conceito de sistemas, não apenas como ferramenta operacional.

Esta aula estabelece esse alicerce. Sem ele, o uso de Docker e Kubernetes se torna mecânico. Com ele, torna-se arquitetural.

MERCADO, CASES E TENDÊNCIAS

A adoção de containers e Kubernetes deixou de ser tendência emergente e virou padrão operacional em times de dados maduros. O principal motivo é econômico: reduzir tempo entre experimento e valor em produção. Em várias empresas, o gargalo não está mais em treinar modelos, sim em implantar, monitorar e manter esses modelos com confiabilidade. Containers entram nesse contexto como unidade de entrega padronizada, permitindo que equipes multidisciplinares (dados, plataforma, segurança e produto) trabalhem sobre contratos técnicos comuns.

Em setores regulados, como financeiro e saúde, cresce o uso de arquiteturas híbridas que combinam isolamento forte em camadas de infraestrutura com portabilidade de aplicação em containers. Essa combinação ajuda a atender requisitos de auditoria e rastreabilidade sem sacrificar velocidade de release. Outro movimento relevante é o aumento de inferência em edge e ambientes distribuídos: a mesma imagem pode ser promovida com ajustes controlados, reduzindo divergências entre ambientes centrais e remotos.

Cases recorrentes mostram ganhos concretos em três frentes. A primeira é previsibilidade: incidentes causados por incompatibilidade de biblioteca caem quando imagens são versionadas e promovidas por pipeline. A segunda é produtividade: o tempo para colocar uma melhoria de modelo em produção reduz quando o empacotamento já nasce padronizado. A terceira é governança: com boas práticas de observabilidade e segurança, as organizações conseguem escalar o número de modelos operando com menor risco operacional.

No curto prazo, as tendências para MLOps apontam para plataformas internas de desenvolvedor (IDP), com templates de serviço de inferência, catálogos de imagens aprovadas e políticas automatizadas de segurança. Também cresce o uso de formatos de empacotamento que preservam metadados de modelo e facilitam reprodução de experimentos. Para quem está se formando agora, entender profundamente containers e virtualização não é diferencial opcional: é base profissional para atuar em projetos de ML com impacto real de negócio.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você construiu o fundamento conceitual que sustenta todo o restante da disciplina. Viu como a virtualização tradicional com hypervisor resolve isolamento em nível de máquina e como containers resolvem isolamento em nível de processo com maior eficiência para ciclos rápidos de engenharia.

Também entendeu o papel de namespaces, cgroups e camadas de filesystem no comportamento dos containers e conectou esses conceitos aos desafios reais de reprodutibilidade e portabilidade em Machine Learning. Com isso, você está pronto(a) para entrar na próxima aula e transformar teoria em prática com Docker.

REFERÊNCIAS

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes**: Up and Running. Sebastopol: O'Reilly Media, 2022.

MERKEL, D. **Docker**: Lightweight Linux Containers for Consistent Development and Deployment. Houston: Belltown Media, 2014.

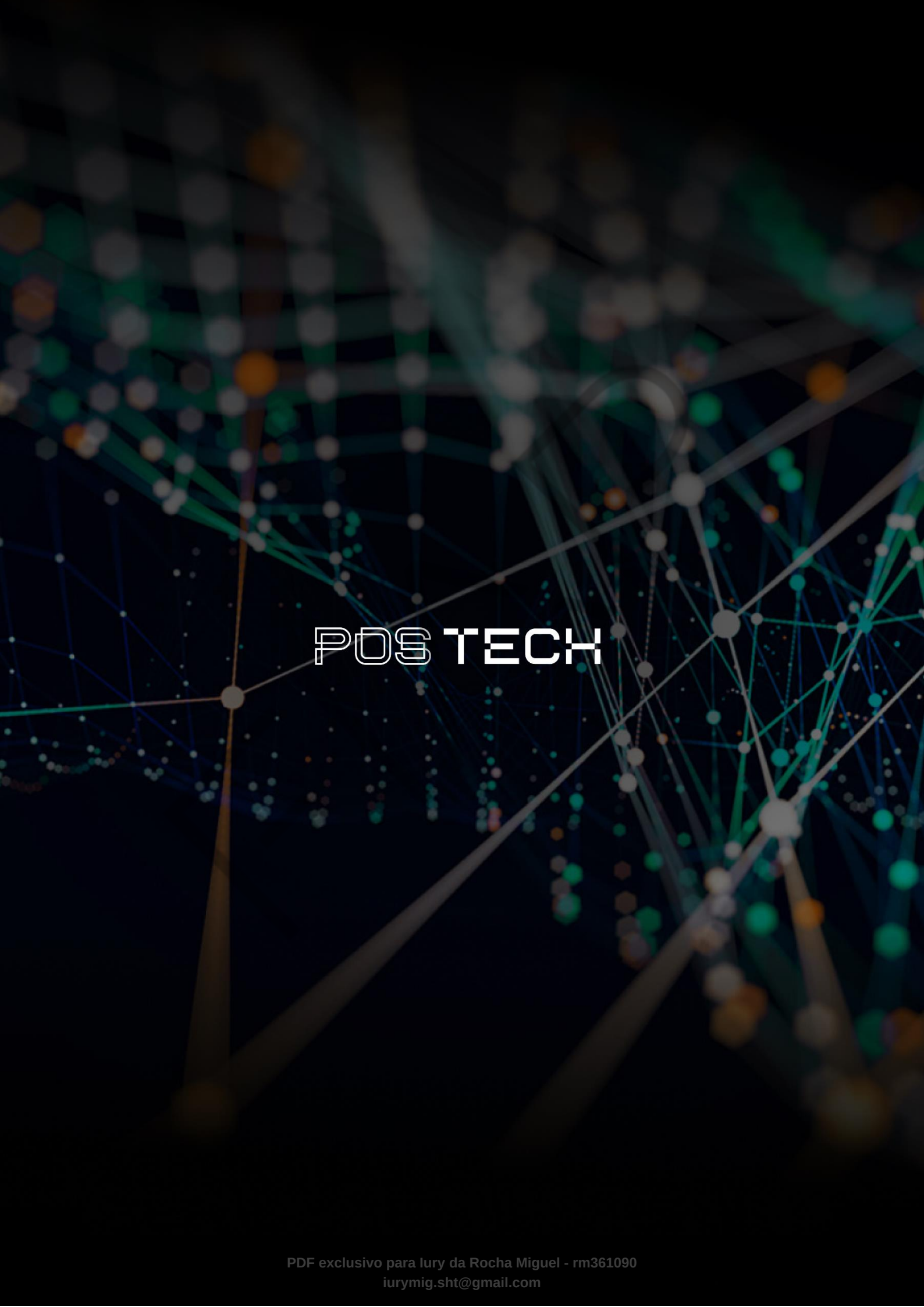
SATO, D. **DevOps na prática**: Entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.

VERAS, M. **Virtualização**: Da teoria a soluções práticas. São Paulo: Novatec, 2019.

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMANIP



POSTECH